Tina Linux 多媒体解码
开发指南

版本号: 1.7
发布日期: 2025.03.29

广州星翼电子科技有限公司zdyzlinux

版本历史

版本号	日期	制/修订人	内容描述
1.0	2019.03.22	AWA0527	初始化 tplayer 相关接口使用说明以及 tplayerdemo 测试用例的使用说明;
1.1	2020.06.12	AWA0527	增加可以只播放音频的配置;
1.2	2021.04.06	AWA1417	更新插图;
1.3	2022.06.28	KPA0567	增加新增接口的使用说明;
1.4	2023.10.13	AWA2047	增加 Tina5.0 的软件配置方法;
1.5	2023.11.20	AWA2047	增加 Tina5.0-buildroot 的软件配置方法;
1.6	2024.11.20	AWA1612	增加 FAQ 章节;
1.7	2025.03.29	AWA12422	增加 cedarx buf 使用说明;

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

目　录

1 概述 **1**
1.1 编写目的 1
1.2 适用范围 1
1.3 相关人员 1

2 软件环境配置 **2**
2.1 内核配置 2
2.1.1 选中 ve 模块 2
2.1.2 选中 ion 模块 3
2.1.3 其他模块 6
2.2 tina 配置 6
2.2.1 选中 tplayer 播放中间件 7
2.2.1.1 openwrt 系统 7
2.2.1.2 buildroot 系统 8
2.2.2 选中 tplayerdemo 8
2.2.2.1 openwrt 系统 8
2.2.2.2 buildroot 系统 10

3 TPlayer 状态图及状态说明 **11**
3.1 TPlayer 状态图 11
3.2 TPlayer 每个状态简要说明 12
3.2.1 Idle 状态 12
3.2.2 Initialized 状态 12
3.2.3 Preparing 状态 12
3.2.4 Prepared 状态 12
3.2.5 Started 状态 12
3.2.6 Paused 状态 13
3.2.7 Stopped 状态 13
3.2.8 PlaybackCompleted 状态 13
3.2.9 Error 状态 13
3.2.10 End 状态 13

4 接口函数说明 **14**
4.1 TPlayerCreate 14
4.2 TPlayerDestroy 14
4.3 TPlayerSetDebugFlag 14
4.4 TPlayerSetNotifyCallback 15
4.5 TPlayerSetDataSource 15
4.6 TPlayerPrepare 15

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.7 TPlayerPrepareAsync 15
4.8 TPlayerStart 16
4.9 TPlayerPause 16
4.10 TPlayerStop 16
4.11 TPlayerReset 16
4.12 TPlayerSeekTo 17
4.13 TPlayerIsPlaying 17
4.14 TPlayerGetCurrentPosition 17
4.15 TPlayerGetDuration 18
4.16 TPlayerGetMediaInfo 18
4.17 TPlayerSetLooping 18
4.18 TPlayerSetScaleDownRatio 18
4.19 TPlayerSetRotate 19
4.20 TPlayerSetSpeed 19
4.21 TPlayerSetVolume 19
4.22 TPlayerGetVolume 19
4.23 TPlayerSetAudioMute 20
4.24 TPlayerSetExternalSubUrl 20
4.25 TPlayerGetSubDelay 20
4.26 TPlayerSetSubDelay 21
4.27 TPlayerGetSubCharset 21
4.28 TPlayerSetSubCharset 21
4.29 TPlayerSwitchAudio 21
4.30 TPlayerSwitchSubtitle 22
4.31 TPlayerSetSubtitleDisplay 22
4.32 TPlayerSetVideoDisplay 22
4.33 TPlayerSetDisplayRect 22
4.34 TPlayerSetSrcRect 23
4.35 TPlayerSetBrightness 23
4.36 TPlayerSetContrast 23
4.37 TPlayerSetHue 23
4.38 TPlayerSetSaturation 24
4.39 TPlayerSetEnhanceDefault 24
4.40 TPlayerGetVideoDispFramerate 24
4.41 TPlayerSetHoldLastPicture 25
4.42 TPlayerSetSoundChannelMode 25
4.43 TPlayerSetG2dRotate 25
4.44 TPlayerSetAudioEQType 25
4.45 TPlayerGetAudioBalance 26
4.46 TPlayerSetAudioBalance 26
4.47 TPlayerGetAudioTrack 26

5 播放器开发流程简单介绍 27

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 播放器开发注意事项 **28**

7 播放器 tplayerdemo 功能测试 **29**

7.1 tplayerdemo 测试用例常用的命令说明 29

7.2 tplayerdemo 测试用例剩余所有的命令说明 31

8 播放器 tplayerdemo 压力测试 **37**

9 FAQ **40**

9.1 seek 跳播不准 40

9.2 cedarx buf 配置说明 40

广州星翼电子科技有限公司zdyzlinux

版权所有 © 珠海全志科技股份有限公司。保留一切权利

插　图

图 2-1 veConfig 3
图 2-2 veConfig for linux5.15 3
图 2-3 ion54Config 4
图 2-4 ionNot34Config 5
图 2-5 ion34Config 5
图 2-6 r30Vp9Config 6
图 2-7 tplayerConfig 7
图 2-8 tplayer5Config 8
图 2-9 tina5.0 buildroot 系统 tplayer 8
图 2-10 tplayerdemoConfig 9
图 2-11 tplayerdemoConfig5 10
图 2-12 buildroot-tplayerdemo config 10
图 3-1 tplayerStatus 11
图 7-1 adbShell 29
图 7-2 tplayerdemo 30
图 7-3 playUrl 30
图 7-4 playUrl 31
图 7-5 setUrl 31
图 7-6 prepare 32
图 7-7 play 32
图 7-8 pause 32
图 7-9 stop 33
图 7-10 seekTo 33
图 7-11 reset 33
图 7-12 quit 34
图 7-13 showMediaInfo 34
图 7-14 showDuration 34
图 7-15 showPosition 35
图 7-16 setLoop 35
图 7-17 setScaleDown 35
图 7-18 fastForward 35
图 7-19 fastBackward 36
图 7-20 getVolume 36
图 7-21 setVolume 36
图 8-1 adbShell 37
图 8-2 tplayerdemoFolder 38
图 8-3 multi_video_play 39

版权所有 © 珠海全志科技股份有限公司。保留一切权利

1 概述

1.1 编写目的

此文档说明从 tina3.0 开始之后的平台，如何使用 TPlayer 的接口来开发播放器应用程序，方便播放器开发人员快速正确地开发，以及播放器测试人员如何根据该文档对 tplayer 播放器进行验证测试。

1.2 适用范围

本文档目前只适用于 tina3.0 之后的平台。硬件平台, 如 R16、R18、R58、R328、R329、R818、R818B、R528、MR813、MR813B、D1、F133、T113、MR527、AI985 等采用 openwrt 构建系统的平台及 T113_S3P、T113_S4P 等采用 buildroot 构建系统的平台均适用。

1.3 相关人员

tina3.0 之后的平台，tplayer 播放器开发和测试人员。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2 软件环境配置

2.1 内核配置

进入内核配置的方式如下：

openwrt 系统：

在 tina 的 sdk 根目录执行 make kernel_menuconfig。

buildroot 系统：

在 tina 的 sdk 根目录执行./build.sh menuconfig

说明

没有 ve 模块的芯片 (如：R328、R329) 平台不需要进行内核配置。

2.1.1 选中 ve 模块

Linux3.10/4.4/4.9/5.4 内核：

```
Device Drivers
  -->Multimedia support
    -->sunxi video encoder and decoder support
```

如下图所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

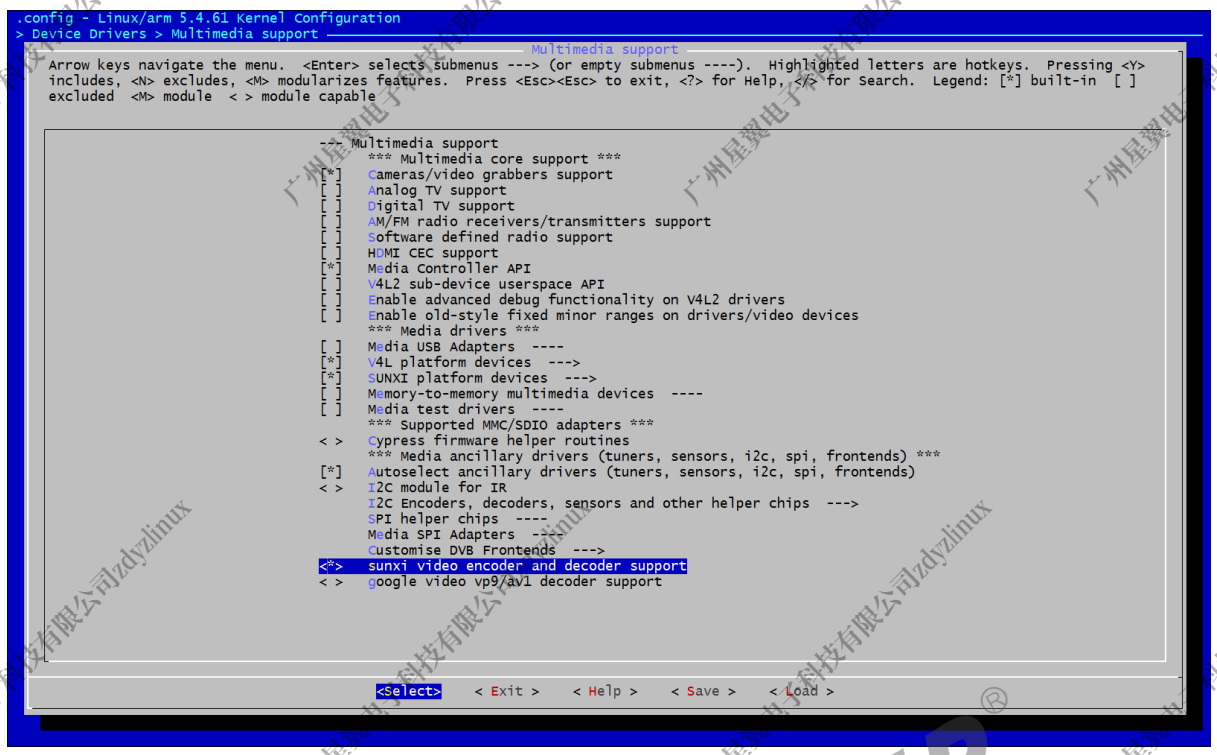

图 2-1: veConfig

Linux5.15 内核：

```
Allwinner BSP
  -->Device Drivers
    -->VE Drivers
      -->sunxi video encoder and decoder support
```

如下图所示：

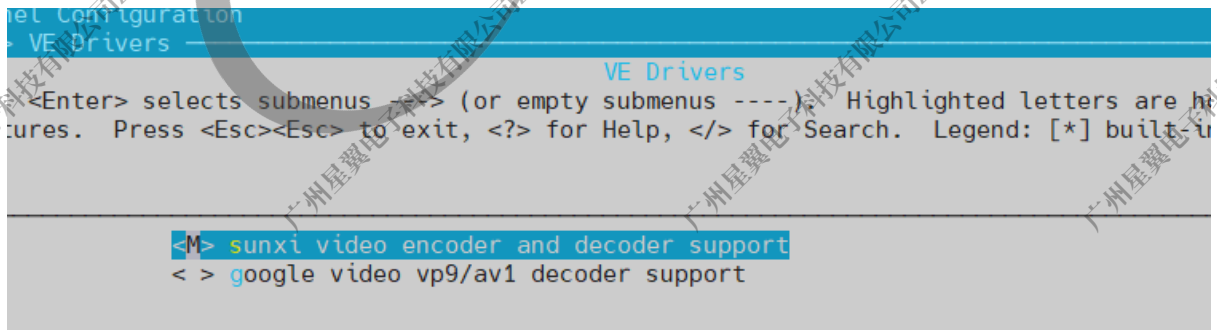

图 2-2: veConfig for linux5.15

2.1.2 选中 ion 模块

linux5.4 内核：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

Device Drivers-->Staging drivers-->Android-->Ion Memory Manager-->Ion system heap
-->Ion CMA heap support

如下图所示：

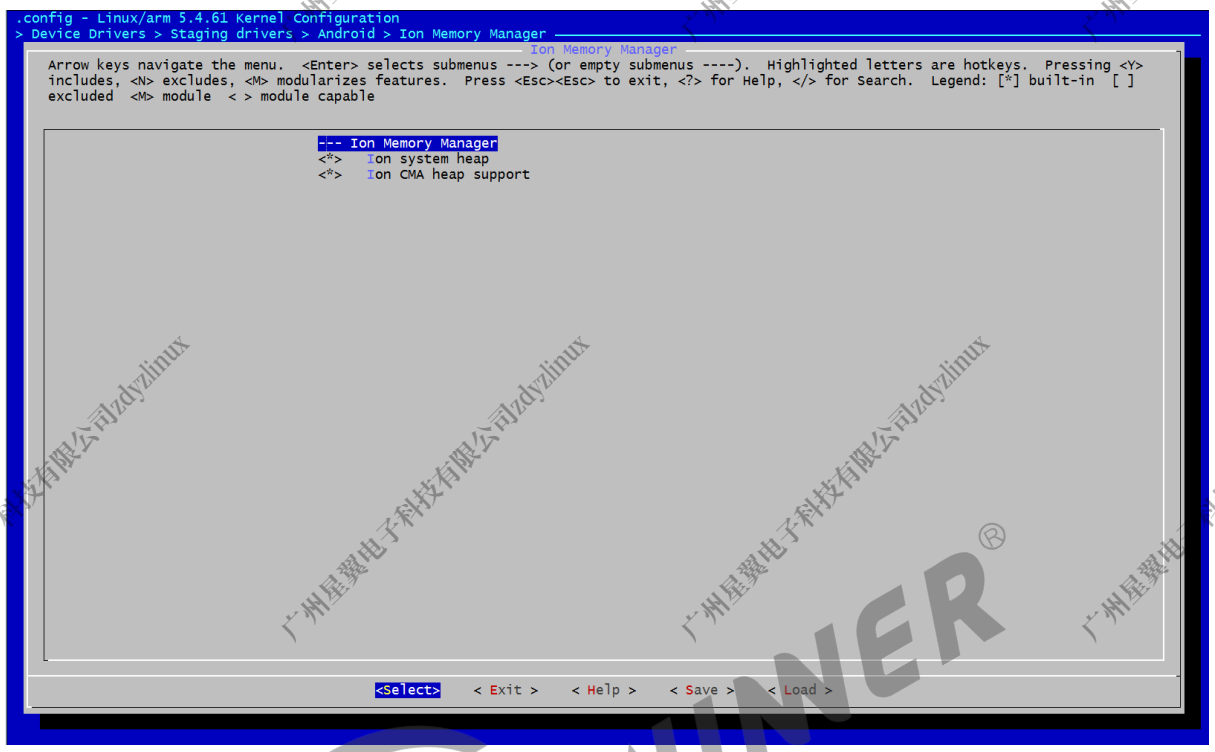

图 2-3: ion54Config

linux3.10/4.4/4.9 内核：

Device Drivers-->Staging drivers-->Android-->Ion Memory Manager-->Ion for SUNXI

如下图所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

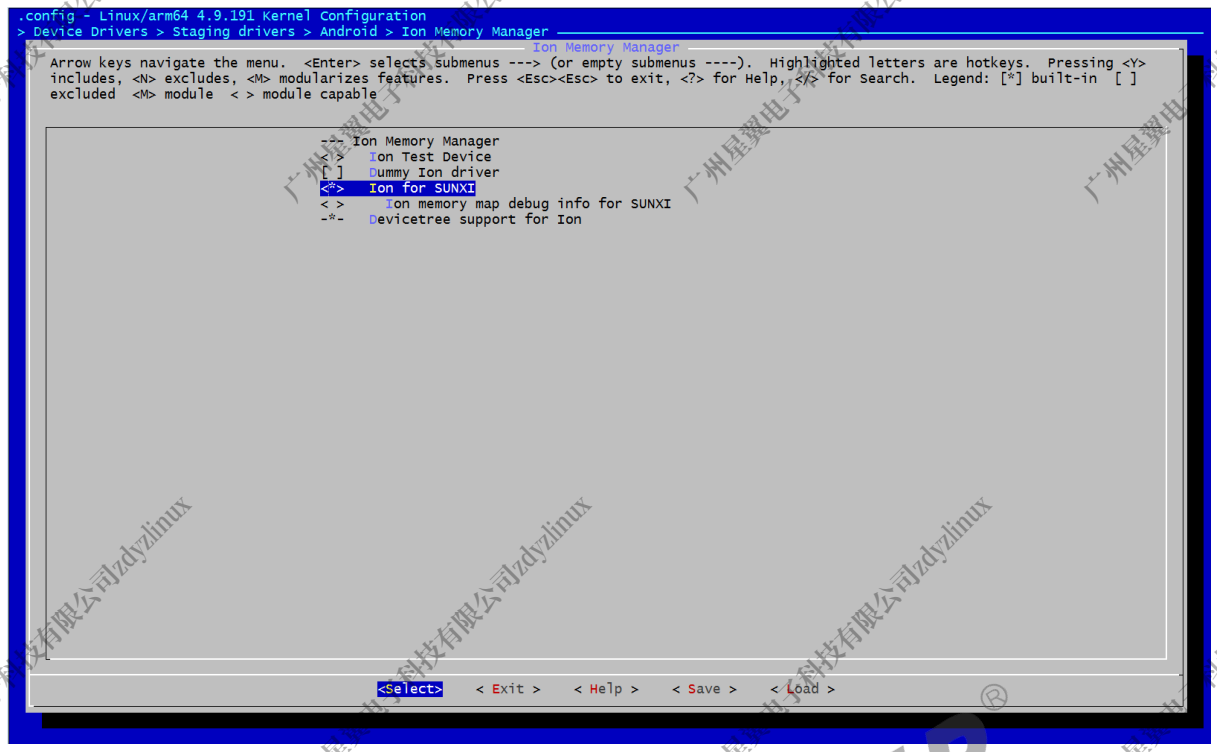

图 2-4: ionNot34Config

linux3.4 内核：

Device Drivers-->Graphics support-->Ion Memory Manager-->ion for sunxi

如下图所示：

图 2-5: ion34Config

版权所有 © 珠海全志科技股份有限公司。保留一切权利

linux5.15：5.15 内核采用的是 dma heap，无需选中 ion。

2.1.3 其他模块

如果是 r30 平台，还需要把 vp9 解码模块选上，否则 vp9 格式不支持。

Device Drivers-->Multimedia support-->google video vp9/av1 decoder support

如下图所示：

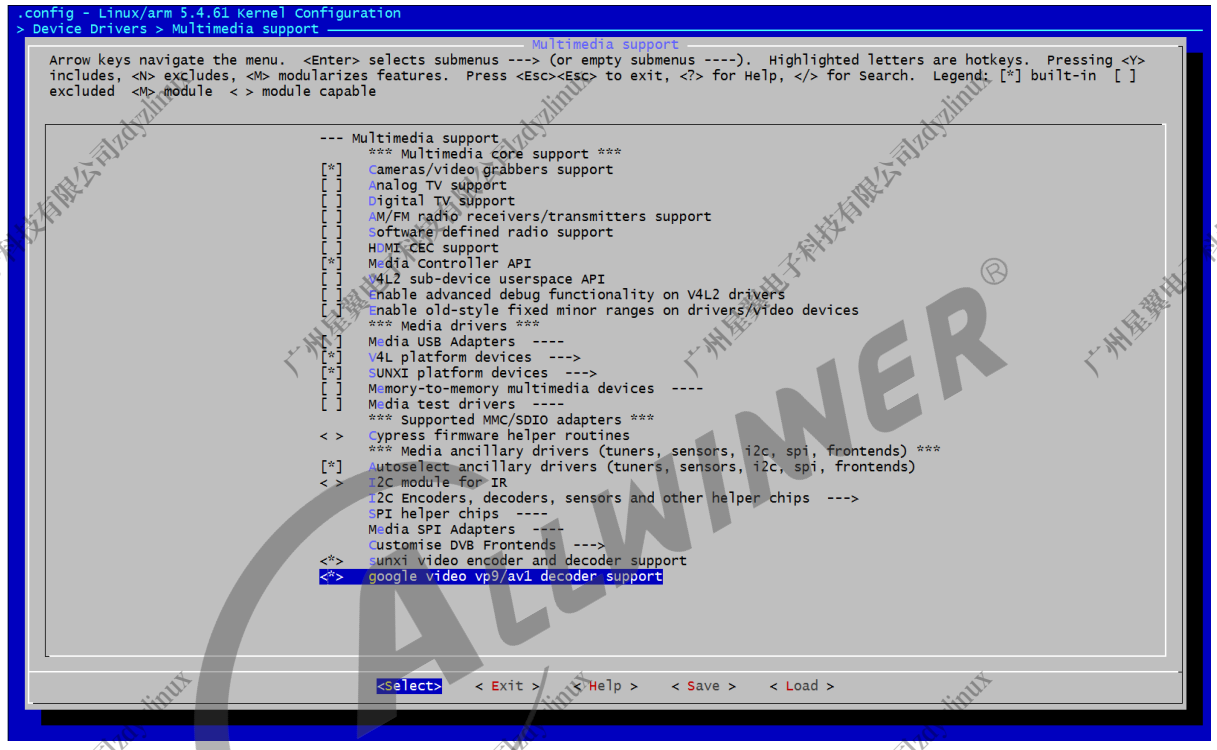

图 2-6: r30Vp9Config

2.2 tina 配置

openwrt 系统：

在 tina 的 sdk 根目录执行 make menuconfig 命令。

buildroot 系统：

在 tina 的 sdk 根目录执行./build.sh buildroot_menuconfig 命令，配置选完后，输入./build.sh buildroot_saveconfig 命令，将配置刷新到 buildroot 的 defconfig 中。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

2.2.1 选中 tplayer 播放中间件

2.2.1.1 openwrt 系统

如果是 libcedarx2.7 版本：

Allwinner-->libcedarx2.7-->Select cedarx configuration options-->Select tplayer middleware

如果是 libcedarx2.8 版本：

Allwinner-->libcedarx-->Select cedarx configuration options-->Select tplayer middleware

如下图所示:

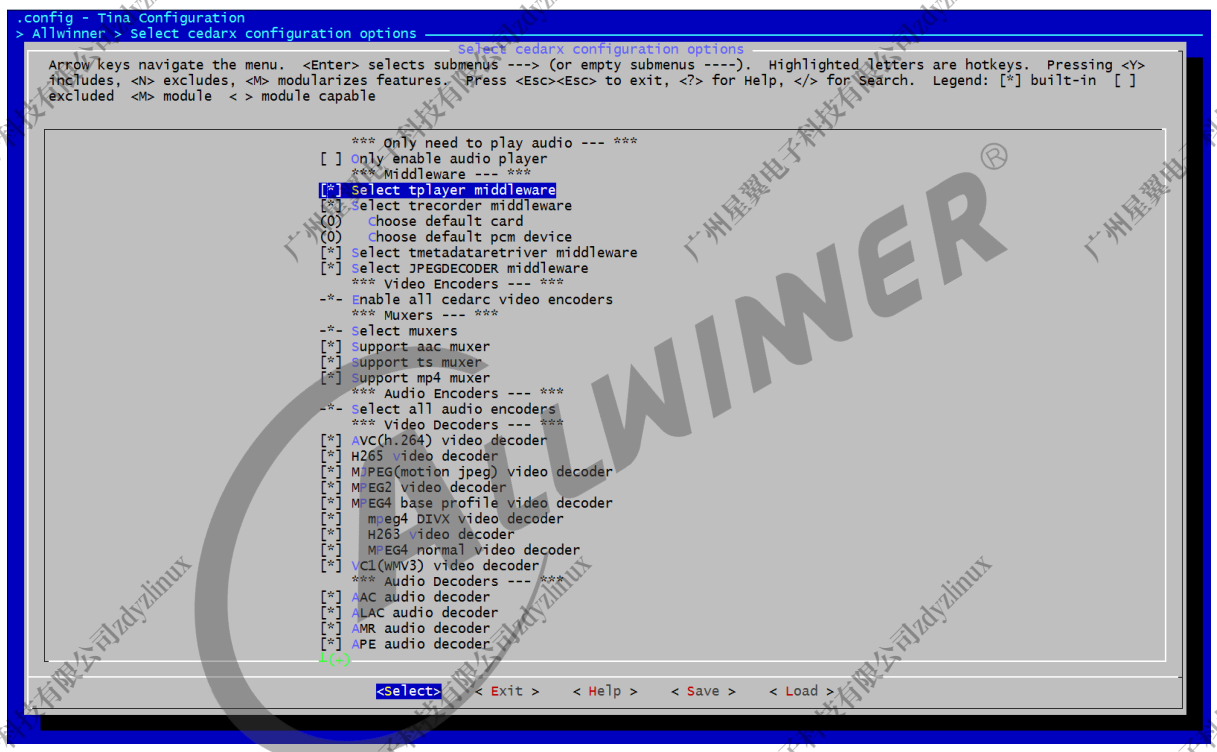

图 2-7: tplayerConfig

如果是 tina5.0：

Allwinner-->Multimedia-->libtmedia-->Select tmedia configuration options-->Select tplayer middleware

如下图所示：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

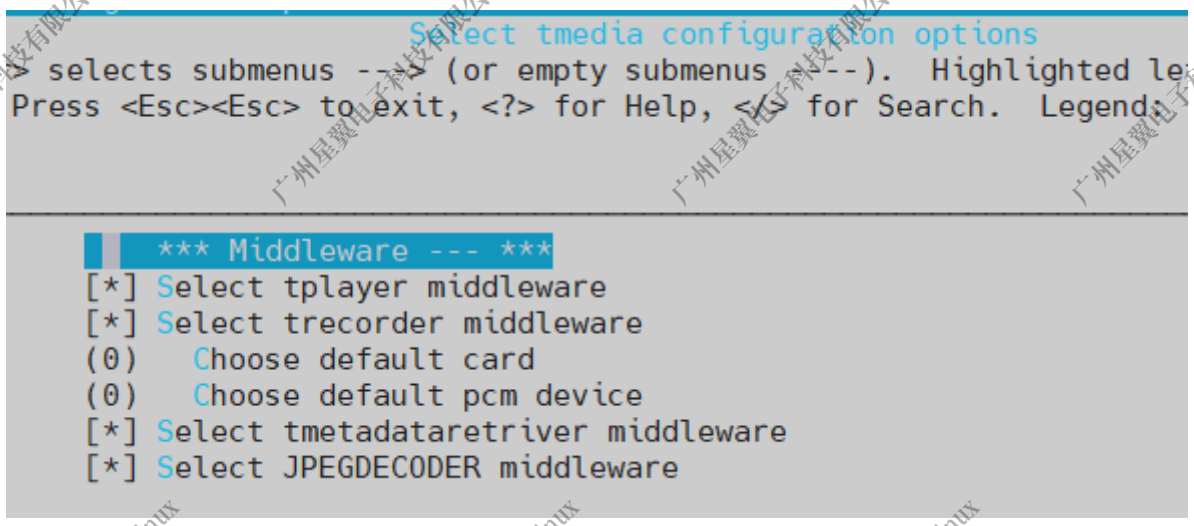

图 2-8: tplayer5Config

2.2.1.2 buildroot 系统

tina5.0-buildroot 系统：

Target packages-->allwinner platform private package select-->cedarx-demo-->libtplayer

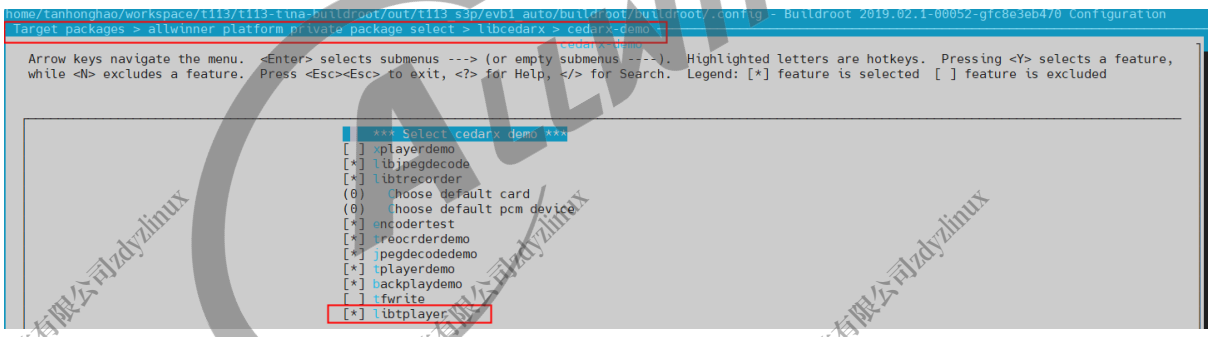

图 2-9: tina5.0 buildroot 系统 tplayer

2.2.2 选中 tplayerdemo

2.2.2.1 openwrt 系统

tina4.x 及以下版本：

Allwinner-->tina_multimedia_demo-->tplayerdemo

版权所有 © 珠海全志科技股份有限公司。保留一切权利

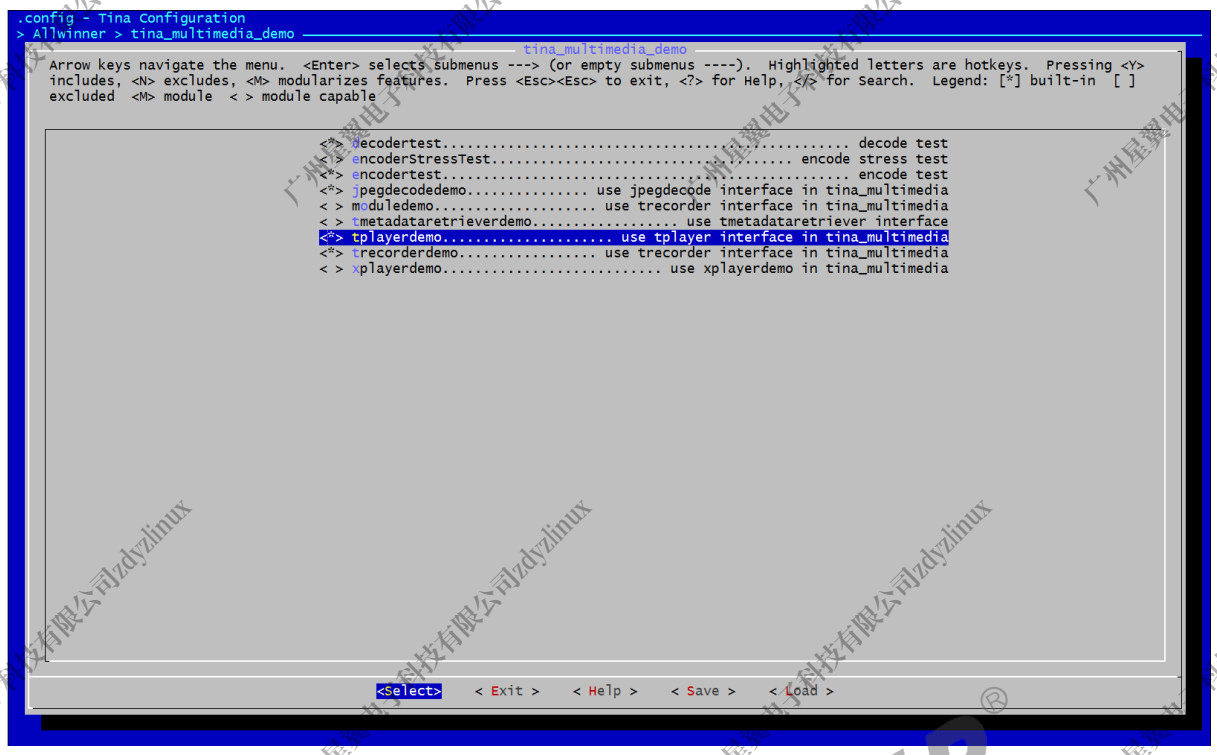

图 2-10: tplayerdemoConfig

tina5.0：

```
Allwinner-->Multimedia-->tplayerdemo
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
                                Multimedia
> selects submenus ---> (or empty submenus ----).  Highlighted letters are hotkeys
Press <Esc><Esc> to exit, <?> for Help, </> for Search.  Legend: [*] built-in  [ ]

    < > backplaydemo..................... use tplayer interface in tina_multimedia
    < > camerapreviewerdemo...... use camerapreviwer interface in tina_multimedia
    <*> decodertest..................................................... decode test
    <*> demo_get_yuv.................................................... demo_get_yuv
    < > encoderStressTest..................................... encode stress test
    <*> encodertest..................................................... encode test
    <*> jpegdecodedemo............... use jpegdecode interface in tina_multimedia
    -*- libcedarc........................................... libcedarc for Allwinner
          Select cedarc configuration options  --->
    < > libcedare........................................... libcedare for Allwinner
    -*- libcedarx........................................... libcedarx  for allwinner
          Select cedarx configuration options  --->
          Select Cedarx memory options  --->
    -*- libtmedia............................... libtmedia  for tplayer and trecoder
          Select tmedia configuration options  --->
          Choose whether to use VIN ISP (YES)  --->
          Choose whether to use playready (NO)  --->
          Choose whether to use opengles keystone correction (NO) ®-->
    < > moduledemo................... use trecorder interface in tina_multimedia
    -*- rt_media.......................................... rt_media for Tina Linux
    < > tmetadataretrieverdemo................... use tmetadataretriever interface
    < > tplayercastdemo..... tplayercastdemo for pdc-pd1 use H line libaw_dlna.so
    <*> tplayerdemo....................... use tplayer interface in tina_multimedia
    <*> trecorderdemo................... use trecorder interface in tina_multimedia
    <*> vencoderDemo..................................... libcedarc encode test
    < > xplayerdemo........................... use xplayerdemo in tina_multimedia
```

图 2-11: tplayerdemoConfig5

2.2.2.2 buildroot 系统

tina5.0-buildroot 系统：

Target packages-->allwinner platform private package select-->cedarx-demo-->tplayerdemo

```
> Target packages > allwinner platform private package select > libcedarx > cedarx-demo
                                                                cedarx-demo
  Arrow keys navigate the menu.  <Enter> selects submenus ---> (or empty submenus ----).  Highlighted letters are hotkeys.
  while <N> excludes a feature.  Press <Esc><Esc> to exit, <?> for Help, </> for Search.  Legend: [*] feature is selected  [

                                  *** Select cedarx demo ***
                                [ ] xplayerdemo
                                [*] libjpegdecode
                                [*] libtrecorder
                                (0)   Choose default card
                                (0)   Choose default pcm device
                                [*] encodertest
                                [*] treocrderdemo
                                [*] jpegdecodedemo
                                [*] tplayerdemo
                                [*] backplaydemo
                                [ ] tfwrite
                                [*] libtplayer
```

图 2-12: buildroot-tplayerdemo config

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3 TPlayer 状态图及状态说明

3.1 TPlayer 状态图

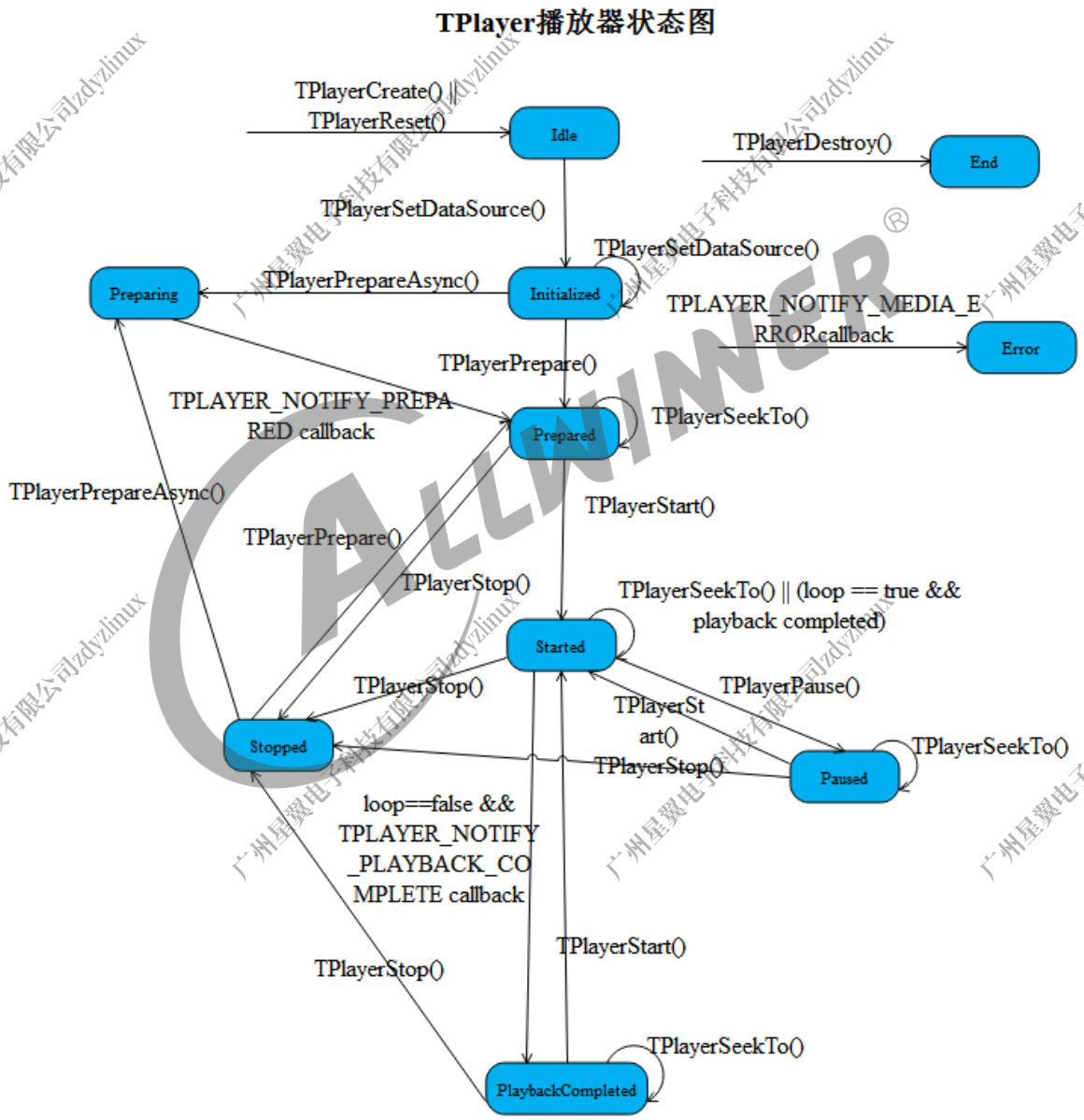

图 3-1: tplayerStatus

这张状态转换图清晰地描述了 TPlayer 的各个状态，也列举了主要方法的调用时序，每种方法只能在一些特定的状态下使用，否则会出错。另外，只有在 Prepared、Started、Paused、Playback-

版权所有 © 珠海全志科技股份有限公司。保留一切权利

Completed 这四种状态下可以进行 TPlayerSeekTo() 操作，并且 TPlayerSeekTo() 之后，状态不变。

3.2 TPlayer 每个状态简要说明

3.2.1 Idle 状态

Idle 状态：当调用 TPlayerCreate() 创建一个 TPlayer 或者调用了其 TPlayerReset() 方法时，TPlayer 处于 idle 状态。

3.2.2 Initialized 状态

这个状态比较简单，调用 TPlayerSetDataSource() 方法就进入 Initialized 状态，表示此时要播放的文件已经设置好了。

3.2.3 Preparing 状态

调用 TPlayerPrepare() 函数还没返回或者是调用 TPlayerPrepareAsync() 并且还没收到 TPLAYER_NOTIFY_PREPARED 这个回调消息的时候就处于 Preparing 状态。

3.2.4 Prepared 状态

调用 TPlayerPrepare() 函数已经返回或者是调用 TPlayerPrepareAsync() 并且已经收到 TPLAYER_NOTIFY_PREPARED 这个回调消息之后的状态就处于 Prepared 状态。在这个状态下说明所有的资源都已经就绪了，调用 TPlayerStart() 函数就可以播放了。

3.2.5 Started 状态

TPlayer 一旦 prepare 完成，就可以调用 TPlayerStart() 方法，这样 TPlayer 就处于 Started 状态，这表明 TPlayer 正在播放文件过程中。可以使用 TPlayerIsPlaying() 测试 TPlayer 是否处于了 Started 状态。如果播放完毕，而又设置了循环播放，则 TPlayer 仍然会处于 Started 状态。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

3.2.6 Paused 状态

Started 状态下可以调用 TPlayerPause() 方法暂停 TPlayer，从而进入 Paused 状态，TPlayer 暂停后再次调用 TPlayerStart() 则可以继续 TPlayer 的播放，转到 Started 状态。

3.2.7 Stopped 状态

Started 或者 Paused 状态下均可调用 TPlayerStop() 停止 TPlayer，而处于 Stop 状态的 TPlayer 要想重新播放，需要通过 TPlayerPrepareAsync() 和 TPlayerPrepare() 回到先前的 Prepared 状态重新开始才可以。

3.2.8 PlaybackCompleted 状态

文件正常播放完毕，而又没有设置循环播放的话就进入该状态，并且会通过 TPLAYER_NOTIFY_PLAYBACK_COMPLETE 这个消息回调给应用。此时可以调用 TPlayerStart() 方法重新从头播放文件，也可以 TPlayerStop() 停止 TPlayer，或者也可以 TPlayerSeekTo() 来重新定位播放位置。

3.2.9 Error 状态

由于某种原因 TPlayer 出现了错误，就会进入该状态，并且会通过 TPLAYER_NOTIFY_MEDIA_ERROR 这个消息回调给应用。如果 TPlayer 进入了 Error 状态，可以通过调用 TPlayerReset() 来恢复，使得 TPlayer 重新返回到 Idle 状态。

3.2.10 End 状态

通过 TPlayerDestroy() 的方法可以进入 End 状态，只要 TPlayer 不再被使用，就应当尽快将其 destroy 掉。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4 接口函数说明

4.1 TPlayerCreate

函数原型	TPlayer* TPlayerCreate(TplayerType type);
功能	创建一个 TPlayer
参数	type: 底层实际对接的播放器类型，有 CEDARX_PLAYER 和 AUDIO_PLAYER 这两种，AUDIO_PLAYER 只能播放音频
返回值	成功返回 TPlayer 的指针，失败返回 NULL
调用说明	无
备注	type 如果是传入 CEDARX_PLAYER，但是也只想播放音频，那么可以在 make menuconfig 的时候选上 Allwinner–>Select cedarx configuration options —> Only enable audio player

4.2 TPlayerDestroy

函数原型	void TPlayerDestroy(TPlayer* p);
功能	销毁一个 TPlayer
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针
返回值	无
调用说明	无

4.3 TPlayerSetDebugFlag

函数原型	int TPlayerSetDebugFlag(TPlayer* p,bool debugFlag);
功能	设置是否打开调试信息
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; debugFlag: 打开调试信息标志,true 为打开,false 为关闭
返回值	成功返回 0，失败返回-1
调用说明	目前该函数还没有实现

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.4 TPlayerSetNotifyCallback

函数原型	int TPlayerSetNotifyCallback(TPlayer* p, TPlayerNotifyCallback notifier, void* pUserData);
功能	设置 TPlayer 的消息回调函数
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; notifier: 回调消息处理函数指针，需要由应用实现; pUserData: 回调消息处理对象
返回值	成功返回 0，失败返回-1
调用说明	创建完 TPlayer 播放器之后，就要调用该函数设置回调消息处理函数。

4.5 TPlayerSetDataSource

函数原型	int TPlayerSetDataSource(TPlayer* p, const char* pUrl,const CdxKeyedVectorT* pHeaders);
功能	设置播放文件的 url，可以是本地文件也可以是网络源
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; pUrl: 需要播放的文件的 url; pHeaders: http 的一些头部信息
返回值	成功返回 0，失败返回-1
调用说明	无

4.6 TPlayerPrepare

函数原型	int TPlayerPrepare(TPlayer* p);
功能	解析文件头部信息，获取元数据
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针
返回值	成功返回 0，失败返回-1
调用说明	该函数是阻塞函数，调用完返回之后就进入了 Prepared 状态，此时可调 TPlayerStart() 函数进行播放

4.7 TPlayerPrepareAsync

函数原型	int TPlayerPrepareAsync(TinaPlayer* p);
功能	解析文件头部信息，获取元数据
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针

版权所有 © 珠海全志科技股份有限公司。保留一切权利

函数原型	int TPlayerPrepareAsync(TinaPlayer* p);
返回值	成功返回 0，失败返回-1
调用说明	该函数是非阻塞函数，需要等到 TPLAYER_NOTIFY_PREPARED 消息回调之后才能调 TPlayerStart() 函数进行播放，而且 TPlayerStart() 函数不能在回调函数中调用

4.8 TPlayerStart

函数原型	int TinaPlayerStart(TPlayer* p);
功能	开始播放
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针
返回值	成功返回 0，失败返回-1
调用说明	无

4.9 TPlayerPause

函数原型	int TPlayerPause(TPlayer* p);
功能	暂停播放
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针
返回值	成功返回 0，失败返回-1
调用说明	无

4.10 TPlayerStop

函数原型	int TPlayerStop(TPlayer* p);
功能	停止播放
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针
返回值	成功返回 0，失败返回-1
调用说明	无

4.11 TPlayerReset

版权所有 © 珠海全志科技股份有限公司。保留一切权利

函数原型	int TPlayerReset(TPlayer* p);
功能	重置播放器
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针
返回值	成功返回 0，失败返回-1
调用说明	在任何状态下都可以调用该函数，每次播放不同的音频之前，都需要调用该函数重置播放器，另外，一般收到 TPLAYER_NOTIFY_MEDIA_ERROR 这个消息的时候，也需要通过调用该函数来重置播放器。但是不能在回调函数中调用该函数，否则会出现死锁

4.12 TPlayerSeekTo

函数原型	int TPlayerSeekTo(TPlayer* p, int nSeekTimeMs);
功能	跳播
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; nSeekTimeMs: 跳播的位置，单位是 ms
返回值	成功返回 0，失败返回-1
调用说明	无

4.13 TPlayerIsPlaying

函数原型	bool TPlayerIsPlaying(TPlayer* p);
功能	是否正在播放
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针
返回值	返回 true 表示正在播放，返回 false 表示没在播放
调用说明	无

4.14 TPlayerGetCurrentPosition

函数原型	int TPlayerGetCurrentPosition(TPlayer* p, int* msec);
功能	获取当前播放的位置
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; msec: 存放当前播放的位置值，单位：ms
返回值	成功返回 0，失败返回-1
调用说明	无

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.15 TPlayerGetDuration

函数原型	int TPlayerGetDuration(TPlayer* p, int* msec);
功能	获取播放的文件总时长
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; msec: 存储文件总时长，单位：ms
返回值	成功返回 0，失败返回-1
调用说明	需要在 prepared 状态之后才可以调用该函数

4.16 TPlayerGetMediaInfo

函数原型	int TPlayerGetMediaInfo(TPlayer* p,MediaInfo* mediaInfo);
功能	获取媒体信息
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; mediaInfo: 存储媒体信息的指针
返回值	成功返回 0，失败返回-1。如果失败，则 mediaInfo 指针为 NULL
调用说明	需要在 prepared 状态之后才可以调用该函数

4.17 TPlayerSetLooping

函数原型	int TPlayerSetLooping(TPlayer* p, bool bLoop);
功能	设置循环播放模式
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; bLoop: true 表示单曲循环，false 表示不会单曲循环
返回值	成功返回 0，失败返回-1。
调用说明	无

4.18 TPlayerSetScaleDownRatio

函数原型	int TPlayerSetScaleDownRatio(TPlayer* p,TplayerVideoScaleDownType nHorizonScaleDown, TplayerVideoScaleDownType nVerticalScaleDown);
功能	设置视频的水平方向的缩放比例和垂直方向的缩放比例
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; nHorizonRatio: 水平方向的缩放比例; nVerticalRatio: 垂直方向的缩放比例
返回值	成功返回 0，失败返回-1

版权所有 © 珠海全志科技股份有限公司。保留一切权利

函数原型	int TPlayerSetScaleDownRatio(TPlayer* p,TplayerVideoScaleDownType nHorizonScaleDown, TplayerVideoScaleDownType nVerticalScaleDown);
调用说明	这个函数需要在 prepare 之前调用

4.19 TPlayerSetRotate

函数原型	int TPlayerSetRotate(TPlayer* p,TplayerVideoRotateType rotateDegree);
功能	设置视频旋转的角度
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; rotateDegree: 视频旋转的角度
返回值	成功返回 0，失败返回-1。
调用说明	这个函数需要在 TPlayerSetDataSource() 函数之前调用

4.20 TPlayerSetSpeed

函数原型	int TPlayerSetSpeed(TPlayer* p, TplayerPlaySpeedType nSpeed);
功能	设置快进快退的速度
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; nSpeed: 快进快退的速度
返回值	成功返回 0，失败返回-1。
调用说明	无

4.21 TPlayerSetVolume

函数原型	int TPlayerSetVolume(TPlayer* p,int volume);
功能	设置播放器的音量大小
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; volume: 需要设置的音量大小值
返回值	成功返回 0，失败返回-1。
调用说明	无

4.22 TPlayerGetVolume

版权所有 © 珠海全志科技股份有限公司。保留一切权利

函数原型	int TPlayerGetVolume(TPlayer* p,int* volume);
功能	获取播放器当前音量的大小
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; volume: 存放获取的音量的大小值
返回值	成功返回 0，失败返回-1。
调用说明	无

4.23 TPlayerSetAudioMute

函数原型	int TPlayerSetAudioMute(TPlayer* p,bool mute);
功能	静音或不静音
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; mute: 是否静音，true 为静音，false 为不静音
返回值	成功返回 0，失败返回-1
调用说明	无

4.24 TPlayerSetExternalSubUrl

函数原型	int TPlayerSetExternalSubUrl(TPlayer* p, const char* filePath);
功能	设置外挂字幕的路径
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; filePath: 外挂字幕的路径
返回值	成功返回 0，失败返回-1。
调用说明	无

4.25 TPlayerGetSubDelay

函数原型	int TPlayerGetSubDelay(TPlayer* p);
功能	获取字幕提前或延时的时间
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针
返回值	成功返回字幕提前或延时的时间，单位：ms，失败返回-1
调用说明	无

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.26 TPlayerSetSubDelay

函数原型	int TPlayerSetSubDelay(TPlayer* p, int nTimeMs);
功能	设置字幕提前或延时的时间
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; nTimeMs: 字幕延时或提前的时间
返回值	成功返回 0，失败返回-1。
调用说明	无

4.27 TPlayerGetSubCharset

函数原型	int TPlayerGetSubCharset(TPlayer* p, char *charset);
功能	获取字幕的字符编码格式
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; charset: 获取字幕的编码格式
返回值	成功返回 0，失败返回-1。
调用说明	无

4.28 TPlayerSetSubCharset

函数原型	int TPlayerSetSubCharset(TPlayer* p, const char* strFormat);
功能	设置字幕的字符编码格式
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; strFormat: 字幕的编码格式
返回值	成功返回 0，失败返回-1。
调用说明	无

4.29 TPlayerSwitchAudio

函数原型	int TPlayerSwitchAudio(TPlayer* p, int nStreamIndex);
功能	切换音轨
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; nStreamIndex: 音轨的 index
返回值	成功返回 0，失败返回-1。
调用说明	无

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.30 TPlayerSwitchSubtitle

函数原型	int TPlayerSwitchSubtitle(TPlayer* p, int nStreamIndex);
功能	切换字幕
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; nStreamIndex: 字幕流的 index
返回值	成功返回 0，失败返回-1。
调用说明	无

4.31 TPlayerSetSubtitleDisplay

函数原型	void TPlayerSetSubtitleDisplay(TPlayer* p, bool onoff);
功能	设置打开或关闭字幕
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; onoff: 打开或关闭字幕的标志位
返回值	无
调用说明	无

4.32 TPlayerSetVideoDisplay

函数原型	void TPlayerSetVideoDisplay(TPlayer* p, bool onoff);
功能	设置是否显示视
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; onoff: 是否显示视频的标志位
返回值	无
调用说明	无

4.33 TPlayerSetDisplayRect

函数原型	void TPlayerSetDisplayRect(TPlayer* p,int x, int y, unsigned int width, unsigned int height);
功能	设置显示的区域
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; x: 显示区域起始点 x 坐标的值; y: 显示区域起始点 y 坐标的值; width: 显示区域的宽; height: 显示区域的高
返回值	无
调用说明	无

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.34 TPlayerSetSrcRect

函数原型	void TPlayerSetSrcRect(TPlayer* p,int x, int y, unsigned int width, unsigned int height);
功能	设置原图像 crop 的坐标和大小
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; x: 源图像 crop 起始点 x 坐标的值; y: 源图像 crop 起始点 y 坐标的值; width: 源图像 crop 的宽; height: 源图像 crop 的高
返回值	无
调用说明	无

4.35 TPlayerSetBrightness

函数原型	void TPlayerSetBrightness(TPlayer* p,unsigned int grade);
功能	设置亮度
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; grade: 亮度的值，范围是:0-100，默认值是:50
返回值	无
调用说明	无

4.36 TPlayerSetContrast

函数原型	void TPlayerSetContrast(TPlayer* p,unsigned int grade);
功能	设置对比度
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; grade: 对比度的值，范围是:0-100，默认值是:50
返回值	无
调用说明	无, 注：这个接口没实现

4.37 TPlayerSetHue

函数原型	void TPlayerSetHue(TPlayer* p,unsigned int grade);
功能	设置色调

版权所有 © 珠海全志科技股份有限公司。保留一切权利

函数原型	void TPlayerSetHue(TPlayer* p,unsigned int grade);
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; grade: 色调的值，范围是:0-100，默认值是:50
返回值	无
调用说明	无, 注：这个接口没实现

4.38 TPlayerSetSaturation

函数原型	void TPlayerSetSaturation(TPlayer* p,unsigned int grade);
功能	设置饱和度
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; grade: 饱和度的值，范围是:0-100，默认值是:50
返回值	无
调用说明	无, 注：这个接口没实现

4.39 TPlayerSetEnhanceDefault

函数原型	void TPlayerSetEnhanceDefault(TPlayer* p);
功能	使能丽色系统，并用默认设置的值
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针
返回值	无
调用说明	无

4.40 TPlayerGetVideoDispFramerate

函数原型	int TPlayerGetVideoDispFramerate(TPlayer* p,float* dispFramerate);
功能	获取显示的帧率
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; dispFramerate: 目前播放实际帧率的值存放在这里
返回值	成功返回 0，失败返回-1。
调用说明	该函数只能在播放过程中调用

版权所有 © 珠海全志科技股份有限公司。保留一切权利

4.41 TPlayerSetHoldLastPicture

函数原型	int TPlayerSetHoldLastPicture(TPlayer* p,int bHoldFlag);
功能	播放完之后是否保留最后一帧画面在屏幕上
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; bHoldFlag: 1 表示需要保留最后一帧，0 表示不需要保留最后一帧
返回值	成功返回 0，失败返回-1。
调用说明	该函数需要在 prepare 之后，start 之前调用

4.42 TPlayerSetSoundChannelMode

函数原型	int TPlayerSetSoundChannelMode(TPlayer* p, int mode);
功能	设置播放器的声道模式
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; mode: 0 表示仅输出左声道，1 表示仅输出右声道，2 表示输出左右声道
返回值	成功返回 0，失败返回-1。
调用说明	该函数需要在 prepare 之后调用

4.43 TPlayerSetG2dRotate

函数原型	int TPlayerSetG2dRotate(TPlayer* p, TplayerVideoRotateType rotate);
功能	使用 G2D 实现视频旋转
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; rotate: 表示旋转度数，目前仅支持 0 度、90 度、180 度、270 度
返回值	成功返回 0，失败返回-1。
调用说明	该函数需要在 prepare 之后，start 之前调用

4.44 TPlayerSetAudioEQType

函数原型	int TPlayerSetAudioEQType(TPlayer* p, AudioEqType type);
功能	设置音频音效
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; type: 0 表示重低音,1 表示流行,2 表示摇滚,3 表示古典,4 表示爵士,5 表示人声,6 表示舞曲,7 表示舒缓
返回值	成功返回 0，失败返回-1。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

函数原型	int TPlayerSetAudioEQType(TPlayer* p, AudioEqType type);
调用说明	该函数需要在 prepare 之后调用

4.45 TPlayerGetAudioBalance

函数原型	int TPlayerGetAudioBalance(TPlayer* p);
功能	获取音频平衡
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针;
返回值	1 表示左声道，2 表示右声道，3 表示立体声，失败返回-1。
调用说明	该函数需要在 prepare 之后调用

4.46 TPlayerSetAudioBalance

函数原型	int TPlayerSetAudioBalance(TPlayer* p, int val);
功能	设置音频平衡
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针; val: 1 表示左声道,2 表示右声道,3 表示立体声
返回值	成功返回 0，失败返回-1。
调用说明	该函数需要在 prepare 之后调用

4.47 TPlayerGetAudioTrack

函数原型	int TPlayerGetAudioTrack(TPlayer* p);
功能	获取音轨序号
参数	p: 通过 TPlayerCreate 函数创建的 TPlayer 指针
返回值	成功返回音频流的序号，失败返回-1。
调用说明	该函数需要在 prepare 之后调用

版权所有 © 珠海全志科技股份有限公司。保留一切权利

5 播放器开发流程简单介绍

（1）TPlayerCreate() //创建一个播放器

（2）TPlayerSetNotifyCallback() //设置消息回调函数

（3）TPlayerSetDataSource() //设置 url

（4）TPlayerPrepare() 或 TPlayerPrepareAsync() //解析头部信息，获取元数据，并根据元数据的信息初始化对应的解码器

（5）TPlayerStart() //播放（注：如果是用 TPlayerPrepareAsync() 函数，则需要等到 TPLAYER_NOTIFY_PREPARED 消息回调之后才可以调用 TPlayerStart() 函数进行播放）

（6）如果需要跳播，则可以调用 TPlayerSeekTo() 函数

（7）如果需要暂停，则调用 TPlayerPause() 函数进行暂停

（8）如果需要停止，则可以调用 TPlayerStop() 或 TPlayerReset() 函数进行停止（注：建议用 TPlayerReset() 函数进行停止，因为任何状态下都可以调用 TPlayerReset() 函数）

（9）如果需要播放下一个或其他的，则可以先调用 TPlayerReset() 函数使播放器进入 idle 状态，然后再重复 (3)(4)(5) 的步骤

（10）详细的播放器开发 demo 可以参考这个路径的内容:
package/allwinner/tina_multimedia_demo/tplayerdemo

版权所有 © 珠海全志科技股份有限公司。保留一切权利

6 播放器开发注意事项

（1）TPlayerSetNotifyCallback(TPlayer* p,TPlayerNotifyCallback notifier,void* pUserData) 函数必需要调用，而且 notifier 不能为 NULL。

（2）回调函数中不能调用 TPlayer 的任何一个接口，如：TPlayerReset、TPlayerStop、TPlayer-Start 等这些接口不能在回调函数中调用。

（3）同一个进程可调用 TPlayerCreate 创建不同的播放器同时播放不同的视频，可通过 TPlayer-SetDisplayRect 设置每个视频的显示区域。但对于同一个视频

channel 不同的 layer，每个视频源的大小和显示区域的缩放比例需要一致，否则显示会有异常。

（4）多视频同时播放，调用 TPlayerDestroy 销毁播放器时，最开始创建的那个播放器需要最后销毁，否则销毁播放器的时候显示会有异常。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

7 播放器 tplayerdemo 功能测试

7.1 tplayerdemo 测试用例常用的命令说明

编译固件之前，如果已经把 tplayerdemo 测试用例选上的话，编译出来的固件就可以使用 tplayerdemo 来测试音视频播放的基本功能了。具体的操作如下：

(1) 把开发板用 usb 线和电脑连接起来，并且确保 adb 能用。如果 adb 不能用，也可以用串口线来代替。

(2) 执行 adb shell 命令：

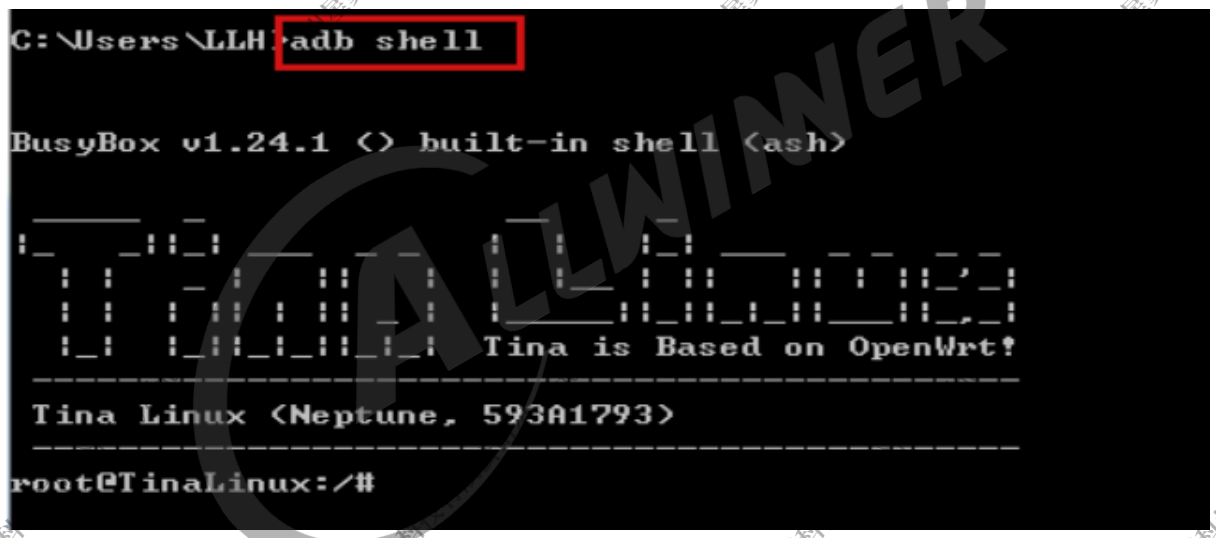

图 7-1: adbShell

(3) 运行 tplayerdemo 测试程序:

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
root@TinaLinux:/# tplayerdemo
tplayerdemo
set MR100 audio throughway
numid=17,iface=MIXER,name='Speaker Function'
  ; type=ENUMERATED,access=rw------,values=1,items=3
  ; Item #0 'headset'
  ; Item #1 'spk'
  ; Item #2 'headset-spk'
  : values=2
numid=1,iface=MIXER,name='Master Playback Volume'
  ; type=INTEGER,access=rw------,values=1,min=0,max=63,step=0
  : values=40
WARNING: awplayer <log_set_level:30>: Set log level to 3
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-0 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-1 ok.
```

图 7-2: tplayerdemo

(4) 播放某个本地或网络音视频：play url: 本地文件的绝对路径或网络 url, 例如需要播放的视频为 h264.mp4，并且该视频放在/mnt/UDISK/目录下，则命令为：

```
tplayerdemo# play url:/mnt/UDISK/h264.mp4
play url:/mnt/UDISK/h264.mp4

tplayerdemo# demoPlayer.mUrl = /mnt/UDISK/h264.mp4WARNING: awplayer <XPlayerReset:946>: reset...
DEBUG  : awplayer <PlayerStop:850>: ****** PlayerStop
ERROR  : awplayer <PlayerStop:855>: ←[40;31minvalid stop operation, player already in stopped status.←[0m
reset the player ok.
DEBUG  : awplayer <XPlayerSetDataSourceUrl:457>: setDataSource(url), url='/mnt/UDISK/h264.mp4'
INFO   : awplayer <XPlayerThread:1731>: process message XPLAYER_COMMAND_SET_SOURCE.
DEBUG  : awplayer <XPlayerPrepare:742>: prepare
DEBUG  : awplayer <XPlayerThread:1984>: process message XPLAYER_COMMAND_PREPARE. mPriData->mStatus: 1
DEBUG  : demuxComponent <DemuxThread:1784>: process message DEMUX_COMMAND_PREPARE.
DEBUG  : demuxComponent <DemuxThread:1851>: === prepare msg
DEBUG  : awplayer <CdxParserPrepare:728>: source uri 'file:///mnt/UDISK/h264.mp4'
DEBUG  : awplayer <__FileStreamCreate:533>: local file 'file:///mnt/UDISK/h264.mp4'
```

图 7-3: playUrl

(5) 注：有一条命令可以直接播放一个本地文件，例如需要播放的视频为 h264_aac_30fps.mp4，并且该视频放在/mnt/UDISK/目录下，则命令为：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
root@TinaLinux:/# tplayerdemo /mnt/UDISK/h264_aac_30fps.mp4
tplayerdemo /mnt/UDISK/h264_aac_30fps.mp4
set r16 audio pass through
numid=1,iface=MIXER,name='headphone volume'
  ; type=INTEGER,access=rw---R--,values=1,min=0,max=63,step=0
  : values=30
  | dBscale-min=-63.00dB,step=1.00dB,mute=0
numid=100,iface=MIXER,name='DACL Mixer AIF1DA0L Switch'
  ; type=BOOLEAN,access=rw------,values=1
  : values=on
numid=96,iface=MIXER,name='DACR Mixer AIF1DA0R Switch'
  ; type=BOOLEAN,access=rw------,values=1
  : values=on
numid=105,iface=MIXER,name='Headphone Switch'
  ; type=BOOLEAN,access=rw------,values=1
  : values=on
WARNING: awplayer <log_set_level:30>: Set log level to 3
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-0 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-1 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-2 ok.
```

图 7-4: playUrl

7.2 tplayerdemo 测试用例剩余所有的命令说明

(1) 执行 set url: 命令设置 url, 其中 set url: 是固定的格式，冒号后面跟本地文件的绝对路径或网络 url 路径:

```
tplayerdemo# set url:/mnt/SDCARD/avc_aac.flv
set url:/mnt/SDCARD/avc_aac.flv

tplayerdemo# demoPlayer.mUrl = /mnt/SDCARD/avc_aac.flvDEBUG  : awplayer <XPlayerSetDataSourceUrl:426>: setDataSource(url), url='/mnt/SDCARD/avc_aac.flv'
INFO   : awplayer <XPlayerThread:1575>: process message XPLAYER_COMMAND_SET_SOURCE.
DEBUG  : awplayer <XPlayerPrepare:686>: prepare
DEBUG  : awplayer <XPlayerThread:1792>: process message XPLAYER_COMMAND_PREPARE. mPriData->mStatus: 1
DEBUG  : demuxComponent <DemuxThread:1693>: process message DEMUX_COMMAND_PREPARE.
DEBUG  : demuxComponent <DemuxThread:1760>: === prepare msg
DEBUG  : awplayer <CdxParserPrepare:711>: source uri 'file:///mnt/SDCARD/avc_aac.flv'
DEBUG  : awplayer <__FileStreamCreate:528>: local file 'file:///mnt/SDCARD/avc_aac.flv'
DEBUG  : awplayer <__FileStreamConnect:387>:     ************impl->size=19835335
DEBUG  : awplayer <__FileStreamConnect:399>: impl->filePath=fd://5?offset=0&length=19835335
DEBUG  : awplayer <__FileStreamConnect:481>: :16:[00 00 00 18 66 74 79 70 69 73 6f 6d 00 00 00 01]
```

图 7-5: setUrl

(2) 执行 prepare 命令解析需要播放的数据:

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
tplayerdemo# prepare
prepare

tplayerdemo# DEBUG  : awplayer <XPlayerPrepareAsync:666>: prepareAsync
DEBUG  : awplayer <XPlayerThread:1792>: process message XPLAYER_COMMAND_PREPARE. mPriData->mStatus: 4
INFO   : awplayer <XPlayerThread:1828>: xxxxxxxxxx video size: width = 640, height = 480
DEBUG : tplayer <CallbackFromXPlayer:82>: video width = 640,height = 480
warning: unknown callback from Tinaplayer.
TPLAYER_NOTIFY_PREPARED,has prepared.
prepare
prepared ok
```

图 7-6: prepare

(3) 执行 play 命令播放。注：如果是播放本地文件，则可以省略 prepare 这个步骤，直接 set url 完成之后，执行 play 命令进行播放。

```
tplayerdemo# play
play

tplayerdemo# DEBUG  : awplayer <XPlayerStart:716>: start
DEBUG  : awplayer <XPlayerThread:1972>: process message XPLAYER_COMMAND_START.
DEBUG  : awplayer <PlayerStart:731>: player start
DEBUG  : awplayer <BaseCompPostAndWait:61>: video decoder receive cmd: start
DEBUG  : awplayer <BaseCompPostAndWait:61>: audio decoder receive cmd: start
error  : AllwinnerAudioCodec <CreateAudioDecoder:1170>: ←[40;31mCreate Decoder!!=====←[0m
error  : AllwinnerAudioCodec <InitCodec:280>: ←[40;31mKhan----Loading 'libaw_aacdec.so' success!←[0m
DEBUG  : awplayer <BaseCompPostAndWait:61>: video render receive cmd: start
DEBUG  : awplayer <BaseCompPostAndWait:61>: audio render receive cmd: start
INFO   : audioRender <handleStart:295>: audio render process start message.
DEBUG  : audioRender <initSoundDevice:478>: init sound device.
```

图 7-7: play

(4) 执行 pause 命令暂停:

```
tplayerdemo# DEBUG  : awplayer <QueueBufferToShow:1247>: video pts(35.002)
pause
pause

tplayerdemo# DEBUG  : awplayer <XPlayerPause:778>: pause
DEBUG  : awplayer <BaseCompPostAndWait:61>: video render receive cmd: pause
DEBUG  : awplayer <BaseCompPostAndWait:61>: audio render receive cmd: pause
INFO   : audioRender <handlePause:363>: audio render process pause message.
DEBUG  : audioRender <handlePause:376>: pause sound devide.
DEBUG : tsoundcontrol <TSoundDevicePause:292>: TinaSoundDevicePause(): sc->sound_status = 0
DEBUG : tsoundcontrol <TSoundDevicePause:303>: alsa can not pause,use snd_pcm_drop
DEBUG  : awplayer <BaseCompPostAndWait:61>: video decoder receive cmd: pause
DEBUG  : awplayer <BaseCompPostAndWait:61>: audio decoder receive cmd: pause
paused.
```

图 7-8: pause

(5) 执行 stop 命令停止:

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
tplayerdemo# stop
stop

tplayerdemo# DEBUG  : awplayer <XPlayerStop:758>: stop
DEBUG  : CdxMovParser <__CdxMovParserForceStop:943>: -- mov ForceStop end
DEBUG  : awplayer <CdxMovClose:5595>: mov close stream = 0x249a0
DEBUG  : awplayer <PlayerStop:843>: ****** PlayerStop
DEBUG  : awplayer <BaseCompPostAndWait:61>: audio render receive cmd: stop
INFO   : audioRender <handleStop:336>: audio render process stop message.
DEBUG  : audioRender <handleStop:349>: stop sound devide.
DEBUG : tsoundcontrol <TSoundDeviceStop:260>: TinaSoundDeviceStop():sc->sound_status = 1
DEBUG : tsoundcontrol <closeSoundDevice:42>: closeSoundDevice()
```

图 7-9: stop

(6) 执行 seek to: 命令进行跳播，其中 seek to: 是固定格式，冒号后面跟跳播的时间，单位是秒。如跳播到 100 秒，则用以下这条命令:

```
tplayerdemo# DEBUG  : awplayer <QueueBufferToShow:1247>: video pts(7.007)
seedebug  : cedarc <savePicture:1349>: saving picture, size: 640 x 480, format: 5, count: 3
DEBUG  : awplayer <QueueBufferToShow:1247>: video pts(8.008)
k toDEBUG  : awplayer <QueueBufferToShow:1247>: video pts(9.009)
:1DEBUG  : awplayer <QueueBufferToShow:1247>: video pts(10.010)
00
seek to:100

tplayerdemo# nSeekTimeMs = 100000 , nDuration = 184617
DEBUG  : awplayer <XPlayerSeekTo:818>: seekTo [100000ms]
DEBUG  : awplayer <XPlayerSeekTo:867>: seek
```

图 7-10: seekTo

(7) 执行 reset 命令重置播放器

```
tplayerdemo# reset
reset

tplayerdemo# WARNING: awplayer <XPlayerReset:887>: reset...
DEBUG  : CdxMovParser <__CdxMovParserForceStop:943>: -- mov ForceStop end
DEBUG  : awplayer <CdxMovClose:5595>: mov close stream = 0xbcc150
DEBUG  : awplayer <PlayerStop:843>: ****** PlayerStop
DEBUG  : awplayer <BaseCompPostAndWait:61>: audio render receive cmd: stop
INFO   : audioRender <handleStop:336>: audio render process stop message.
DEBUG  : audioRender <handleStop:349>: stop sound devide.
DEBUG : tsoundcontrol <TSoundDeviceStop:260>: TinaSoundDeviceStop():sc->sound_status = 1
DEBUG : tsoundcontrol <closeSoundDevice:42>: closeSoundDevice()
ERROR : tsoundcontrol <closeSoundDevice:52>: ←[40;31malsa-uninit: pcm closed←[0m
DEBUG  : awplayer <BaseCompPostAndWait:61>: video render receive cmd: stop
DEBUG  : awplayer <__LayerCtrlHoldLastPicture:368>: LayerCtrlHoldLastPicture, bHold = 0
DEBUG  : awplayer <BaseCompPostAndWait:61>: audio decoder receive cmd: stop
debug  : AllwinnerAudioCodec <ExitCodec:329>: Khan----dlclose libaw_aacdec.so success!
error  : AllwinnerAudioCodec <DestroyAudioDecoder:1090>: ←[40;31mdestroy_ResampleInfo fail!!←[0m
DEBUG  : awplayer <BaseCompPostAndWait:61>: video decoder receive cmd: stop
DEBUG  : awplayer <PlayerStop:875>: ****** PlayerStop end
DEBUG  : awplayer <BaseCompPostAndWait:61>: video render receive cmd: quit
```

图 7-11: reset

版权所有 © 珠海全志科技股份有限公司。保留一切权利

(8) 执行 quit 命令退出 tplayerdemo 程序或者直接 ctrl+c 退出 tplayerdemo 程序

```
tplayerdemo# quit
quit

tplayerdemo# COMMAND_QUIT
destroy TinaPlayer.
WARNING: awplayer <XPlayerDestroy:297>: XPlayerDestroy
WARNING: awplayer <XPlayerReset:887>: reset...
```

图 7-12: quit

(9)show media info 这条命令可以打印一些媒体信息出来。注：这条命令需要在 prepare 之后运行才有效

```
tplayerdemo# show media info
show media info

tplayerdemo# show media information:
file size = 19370 KB
duration = 184617 ms
bitrate = 839 Kbps
container type = 0
video stream num = 1
audio stream num = 1
subtitle stream num = 0
video codec tpye = 277
video width = 640
video height = 480
video framerate = 29970
video frameduration = 0
audio codec tpye = 4
audio channel num = 2
audio BitsPerSample = 16
audio sample rate = 44100
audio bitrate = 0 Kbps
```

图 7-13: showMediaInfo

(10)show duration 这条命令可以打印总时长。注：这条命令需要在 prepare 之后运行才有效

```
tplayerdemo# show duration
show duration

tplayerdemo# media duration = 184 seconds.
```

图 7-14: showDuration

(11)show position 这条命令可以打印当前播放到的时间点。注：这条命令需要在 prepare 之后运行才有效

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
tplayerdemo# show position
show position

tplayerdemo# current position = 6 seconds.
```

图 7-15: showPosition

(12)set loop 这条命令可以设置是否是单曲循环，如果需要单曲循环，则设为:set loop:1，如果不想单曲循环，则设为：set loop:0 . 注：默认不设置的话是为 0 的

```
tplayerdemo# set loop:1
set loop:1

tplayerdemo# tplayerdemo set loop flag:flag = 1
```

图 7-16: setLoop

(13)set scaledown 这条命令可以设置视频解码后的数据需要缩放的比例，目前支持不缩放、缩放 1/2 和 1/4，jpeg 格式支持缩放到 1/8。如要缩放为原来的 1/2，则用以下命令。注：这个命令需要在 set url 命令前调用

```
tplayerdemo# set scaledown:2
set scaledown:2

tplayerdemo# tplayerdemo set scaledown value = 2
scale down 1/2
```

图 7-17: setScaleDown

(14)fast forward 这条命令可以快进。目前支持 2、4、8、16 倍快进，如需要 4 倍快进，则用以下这条命令：

```
tplayerdemo# DEBUG  : awplayer <QueueBufferToShow:1247>: video pts(75.008)
fast forward:4
fast forward:4

tplayerdemo# tplayerdemo fast forward times = 4
fast forward 4 times
```

图 7-18: fastForward

(15)fast backward 这条命令可以快退。目前支持 2、4、8、16 倍快退，如需要 4 倍快退，则用以下这条命令：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
tplayerdemo# DEBUG  : awplayer <QueueBufferToShow:1247>: video pts(104.004)
fast backward:4DEBUG  : awplayer <QueueBufferToShow:1247>: video pts(105.005)

fast backward:4

tplayerdemo# tplayerdemo fast backward times = 4
fast backward 4 times
```

图 7-19: fastBackward

(16)get volume 这条命令可以获取播放器的音量值。

```
tplayerdemo# get volume
get volume

tplayerdemo# cur volume = 20
```

图 7-20: getVolume

(17)set volume 这条命令可以设置播放器的音量值，支持的音量值范围:0-40，其中 0 表示静音。如：要把播放器音量值设为 30，则可以用下面这条命令：

```
tplayerdemo# set volume:30
set volume:30

tplayerdemo# tplayerdemo setVolume:volume = 30
tplayerdemo set volume ok
```

图 7-21: setVolume

版权所有 © 珠海全志科技股份有限公司。保留一切权利

8 播放器 tplayerdemo 压力测试

编译固件之前，如果已经把 tplayerdemo 测试用例选上的话，编译出来的固件就可以使用 tplayerdemo 来音视频播放进行压力测试了。具体的操作如下：

(1) 把开发板用 usb 线和电脑连接起来，并且确保 adb 能用。如果 adb 不能用，也可以用串口线来代替。

(2) 执行 adb shell 命令：

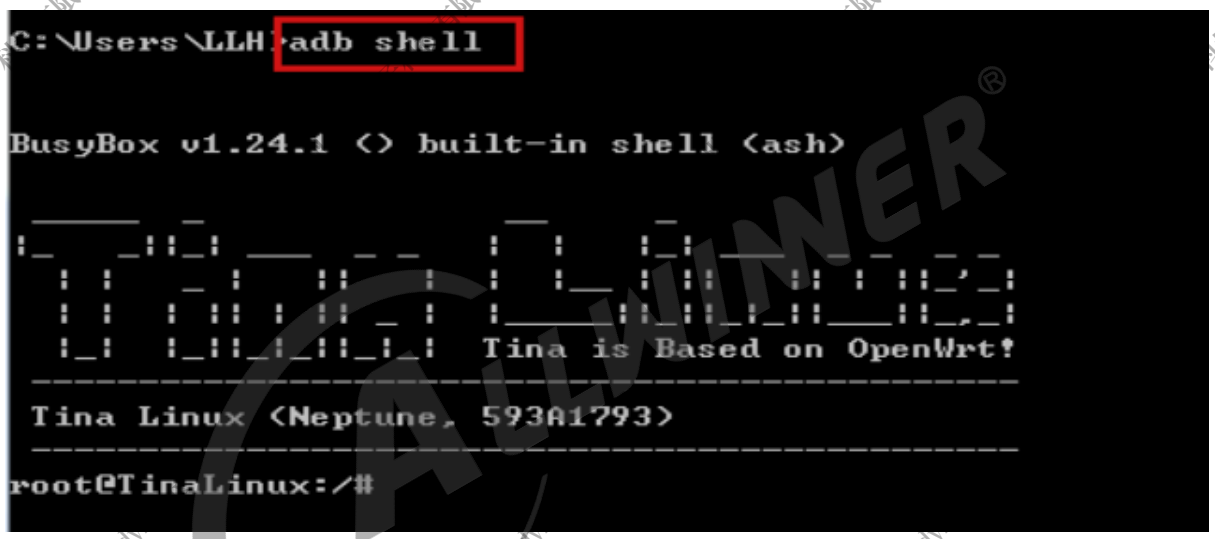

图 8-1: adbShell

(3) 假设需要循环测试某个目录下的所有音视频文件，则可以用这条命令：tplayerdemo 音视频文件所在的文件夹的绝对路径，假如需要测试的音视频文件放在/mnt/UDISK/目录下，则运行以下命令则可以循环播放/mnt/UDISK/目录下的所有音视频文件：

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
root@TinaLinux:/# tplayerdemo /mnt/UDISK/
tplayerdemo /mnt/UDISK/
set r16 audio pass through
numid=1,iface=MIXER,name='headphone volume'
  ; type=INTEGER,access=rw---R--,values=1,min=0,max=63,step=0
  : values=30
  | dBscale-min=-63.00dB,step=1.00dB,mute=0
numid=100,iface=MIXER,name='DACL Mixer AIF1DA0L Switch'
  ; type=BOOLEAN,access=rw------,values=1
  : values=on
numid=96,iface=MIXER,name='DACR Mixer AIF1DA0R Switch'
  ; type=BOOLEAN,access=rw------,values=1
  : values=on
numid=105,iface=MIXER,name='Headphone Switch'
  ; type=BOOLEAN,access=rw------,values=1
  : values=on
WARNING: awplayer <log_set_level:30>: Set log level to 3
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-0 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-1 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-2 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-3 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-4 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-5 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-6 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-7 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-8 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-9 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-10 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-11 ok.
DEBUG  : awplayer <ReadPluginEntry:178>: read plugin entry adecoder-12 ok.
```

图 8-2: tplayerdemoFolder

(4) 多视频循环压测命令：tplayerdemo 每个视频的绝对路径。假设需要同时压测 4 个视频，并且这 4 个文件放在/mnt/UDISK/目录下，则可运行以下命令进行压测。注：对于只有一个视频 channel 的平台，因为最多只有 4 个 layer，所以最多只支持 4 个视频同时播放。

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
root@TinaLinux:/# tplayerdemo /mnt/UDISK/h264_1_10s_400_240_ok.avi /mnt/UDISK/h2
64_2_10s_400_240_ok.mp4 /mnt/UDISK/h265_1_10s_400_240_ok.mp4 /mnt/UDISK/h265_3_1
2s_400_240_ok.mp4
WARNING: awplayer <cdx_log_set_level:30>: cdx Set log level to 6
INFO   : cedarc <CedarPluginVDInit:79>: register h264 decoder success!
INFO   : cedarc <CedarPluginVDInit:84>: register mjpeg decoder success!
INFO   : cedarc <CedarPluginVDInit:86>: register mpeg2 decoder success!
INFO   : cedarc <CedarPluginVDInit:92>: register mpeg4dx decoder success!
INFO   : cedarc <CedarPluginVDInit:79>: register mpeg4H263 decoder success!
INFO   : cedarc <CedarPluginVDInit:90>: register mpeg4Normal decoder success!
INFO   : cedarc <CedarPluginVDInit:74>: register vc1 decoder success!
INFO   : cedarc <CedarPluginVDInit:85>: register h265 decoder success!

*****************************************************************************************
* This program implements a simple player, you can type commands to control the player.
* To show what commands supported, type 'help'.
*****************************************************************************************
dd: writing '/dev/fb0': No space left on device
6001+0 records in
6000+0 records out
argc = 5
argv[0] = tplayerdemo
argv[1] = /mnt/UDISK/h264_1_10s_400_240_ok.avi
argv[2] = /mnt/UDISK/h264_2_10s_400_240_ok.mp4
argv[3] = /mnt/UDISK/h265_1_10s_400_240_ok.mp4
argv[4] = /mnt/UDISK/h265_3_12s_400_240_ok.mp4
may be is one file:cut down suffix is:.avi
find the matched type:.avi
create player:0

>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>>> tina_multimedia <<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<
tag    : tina3.5
branch: tina-dev
date   : Mon Jul 15 19:04:59 2019 +0800
Change-Id: I5f6c8a88d7b387a312b7744797a0d5f8ab07ee7a
-----------------------------------------------------------------------------------
```

图 8-3: multi_video_play

版权所有 © 珠海全志科技股份有限公司。保留一切权利

9 FAQ

9.1 seek 跳播不准

使用 TPlayerSeekTo 函数跳转到例如 23 秒，但实际出现的不是 23 秒的视频；Seek 操作只能跳到关键帧 I 帧，若 Seek 的位置不在关键帧上，则会跳到相邻的 I 帧上; 比如当前 seek 到 23 秒；第 23.000000 是个 B 帧，下一个 I 帧位于 32.840000；此时 seek 的实际位置是 32.840000;

解决方法：使用 ffmpeg 将 ai.mkv 转换，当前帧率是 25fps，每个整数秒插入一个 I 帧。

```
sudo ffmpeg -i ai.mkv -c:v libx265 -preset fast -x265-params keyint=25:min-keyint=25 test_key25.mkv

参数解析：
-i ai.mkv：输入的原始视频文件。
-c:v libx265：使用 H.265 编码器（libx265）重新编码视频。
-g 25：设置 GOP 长度为 25，意味着每 25 帧插入一个 I 帧（适用于每秒 25 帧的帧率）。
-x265-params keyint=25:min-keyint=25：强制设置每 25 帧插入一个 I 帧。keyint 设置了最大 GOP 长度，min-keyint 强制在
        每个 GOP 内至少插入一个 I 帧。
-preset fast：选择编码速度的预设。你可以选择 ultrafast、fast、medium、slow 等，根据你对速度和压缩效率的需求。
test_key25.mkv：输出的视频文件
```

对应的 FAQ 链接：

```
FAQ3155:

https://one.allwinnertech.com/#/faq/0/show?showType=edit&faqId=3155
```

9.2 cedarx buf 配置说明

```
[paramter]
//开始播放前需缓冲的视频帧数
start_play_cache_video_frame_num = 30
//开始播放前需缓冲的媒体码流数据大小
start_play_cache_size = 128          # KB
//缓冲线程缓冲数据 buffer 的大小，点播(vod类型资源)
cache_buffer_size = 20480            # KB
//缓冲线程缓冲数据 buffer 的大小，直播(live类型资源)
cache_buffer_size_live = 20480       # KB
//缓冲一次可以播放的时长
start_play_cache_time = 5            # Second
//可设置的最大的开始播放前需缓冲的媒体码流数据大小
max_start_play_chache_size = 153600   # KB
//可设置的最大的缓冲线程缓冲数据 buffer 的大小
max_cache_buffer_size = 204800       # KB

//显示端需要hold住的buf个数，至少为2
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

```
pic_4list_num = 3
//interlace片源的视频会用到，设置为2 注：如果不是interlace片源的视频，那么该buf不会使用
pic_4di_num = 2
//旋转模块占用显存个数
pic_4rotate_num = 0
//平滑显示的buf，至少设置为1，否则会引起卡顿
pic_4smooth_num = 3

//切换视频的时候，所有fbm buf都要被释放，为了避免切换过程出现黑屏情况，会申请一块buf出来保存最后一帧视频，为兼
    容所有分辨率的视频，这块buf的宽一般设置为该产品支持的最高分辨率的宽，需要64对齐
hold_last_picture_with = 1920
hold_last_picture_height = 1088
```

版权所有 © 珠海全志科技股份有限公司。保留一切权利

著作权声明

版权所有 ©2025 珠海全志科技股份有限公司。保留一切权利。

本文档及内容受著作权法保护，其著作权由珠海全志科技股份有限公司（“全志”）拥有并保留一切权利。

本文档是全志的原创作品和版权财产，未经全志书面许可，任何单位和个人不得擅自摘抄、复制、修改、发表或传播本文档内容的部分或全部，且不得以任何形式传播。

商标声明

ALLWINNER、ALLWINNER 全志科技、全志科技、ALLWINNER（不完全列举）均为珠海全志科技股份有限公司的商标或者注册商标。在本文档描述的产品中出现的其它商标，产品名称，和服务名称，均由其各自所有人拥有。

免责声明

您购买的产品、服务或特性应受您与珠海全志科技股份有限公司（“全志”）之间签署的商业合同和条款的约束。本文档中描述的全部或部分产品、服务或特性可能不在您所购买或使用的范围内。使用前请认真阅读合同条款和相关说明，并严格遵循本文档的使用说明。您将自行承担任何不当使用行为（包括但不限于如超压，超频，超温使用）造成的不利后果，全志概不负责。

本文档作为使用指导仅供参考。由于产品版本升级或其他原因，本文档内容有可能修改，如有变更，恕不另行通知。全志尽全力在本文档中提供准确的信息，但并不确保内容完全没有错误，因使用本文档而发生损害（包括但不限于间接的、偶然的、特殊的损失）或发生侵犯第三方权利事件，全志概不负责。本文档中的所有陈述、信息和建议并不构成任何明示或暗示的保证或承诺。

本文档未以明示或暗示或其他方式授予全志的任何专利或知识产权。在您实施方案或使用产品的过程中，可能需要获得第三方的权利许可。请您自行向第三方权利人获取相关的许可。全志不承担也不代为支付任何关于获取第三方许可的许可费或版税（专利税）。全志不对您所使用的第三方许可技术做出任何保证、赔偿或承担其他义务。